Secure By Design

EDUCATE. EMPOWER. SECURE THE FUTURE.

COURSE 1.11 / COMPANION WORKBOOK

Protect the record on SBD-AUDIT01

Keep the audit record intact, even from its administrators

The second of three logging courses: build SBD-AUDIT01 as the protected home for the collector's record — daily archives over encrypted SMB from a write-only managed account, SHA-256 hashes cataloged on arrival and verified every day, access auditing that leaves the box, retention that runs only as an approved and logged process, and proof that an administrator's delete is refused and a tampered copy is caught.

MATERIALS EDITION 1.0 / COURSE 1.11

Updated 14 October 2026 · sbd-audit-repository

NAME _____ / DATE _____

© 2026 Secure By Design Team LLC.

START HERE / YOUR LEARNING PATH

Table of contents

Read each lesson alongside the interactive guide and the video. Use the record tables to write down what you actually observe — the workbook is your evidence, not the guide.

Before you start

01 Design the repository and build SBD-AUDIT01

02 Archive, hash, and protect the record

03 Verify the catalog and watch the repository

04 Run retention and prove retrieval

05 Prove the record survives and hand off

Course review and sign-off

> **HOW TO USE IT** Each lesson page has the same shape: what you will learn, the build steps, the commands that verify them, and a record table. Fill the table from your own environment — never from the sample values.

Before you start

This is the second of three logging courses: course 1.10 collects the record on SBD-SYSLOG01, this course gives it a protected home on SBD-AUDIT01, and course 1.12 analyzes a working copy in SBD-SPLUNK01. Protection is something you can show: one write path, permissions that refuse even administrators, a hash for every archive, and retention that runs only as an approved, logged process. How long each stage keeps the record depends on your environment and policy: the course says what each stage keeps and why, and you record the periods you chose.

Prerequisites

- ☐ Complete course 1.10: SBD-SYSLOG01 collects Windows events into the ForwardedEvents log (on D:\Logs\ForwardedEvents, overwriting the oldest) and syslog into daily files per sender under D:\Logs\Syslog, keeps the buffer period course 1.10 recorded, and the register carries row F11 waiting for this course.
- ☐ Provision SBD-AUDIT01 (10.0.50.40, VLAN 50) as Windows Server with a separate data volume (D: in this sample), ready to join sbd.test, with the hypervisor's time synchronization turned off; lesson 1 sizes the volume.
- ☐ Have course 1.8's administration seats in place — the audit custodian's separate accounts in SBD Audit Custodians work from SBD-ADMIN-SRV01 (rule A04), and directory changes are Tier 0 work from AdminJumpBox01 — plus the Tier 1 SCC copy course 1.8 installed on SBD-ADMIN-SRV01, at course 1.4's tool and benchmark versions, for the first baseline.

What this course builds

- Design and build the repository: retention, capacity, one write path, and the custodian's separate administration.
- Archive the collector's streams daily over encrypted SMB from a write-only gMSA, with every archive hashed and cataloged.
- Verify the catalog daily, audit access to the repository, and run retention as an approved, logged process.
- Prove an administrator's delete is refused and logged, a tampered copy is caught, and an old archive can still be read.

Systems in scope

RECORD	YOUR OBSERVATION
SBD-AUDIT01 — name, address, OS build	
SBD-SYSLOG01 — name, address, OS build	

LESSON 01 / ABOUT 30 MIN

Design the repository and build SBD-AUDIT01

> **WHY THIS MATTERS** The repository has one job: keep the collector's record intact for its whole retention period, even from its own administrators. Size it, fix the one way in, name every identity and what it may do, and build SBD-AUDIT01 where the model puts it — before a single archive arrives.

In this lesson you will

- Size the repository for the retention period you record and the volume the collector actually receives.
- Define the single write path and the identities that may write, read, and delete.
- Build SBD-AUDIT01 in its own OU, administered only by the audit custodian.

Follow the build

1. Write the repository's rules from course 1.10's design (AU-11): the collector buffers long enough to cover a missed archive night and a source replaying its backlog; the repository keeps the protected copy for the whole retention period your records policy requires; and analysis keeps a shorter searchable window for investigations. Each period depends on your environment and policy — record the values you chose and who approved them. The repository holds the protected copy and nothing else — it is not a search tool, and nothing is removed from it except by the custodian's approved, logged retention process.
2. Size it (AU-4): on SBD-SYSLOG01, export the last 24 hours of ForwardedEvents to a temporary file and add up the syslog files written in the same day. Multiply a typical day by the days in your recorded retention period, add headroom for growth (the fleet joins in course 1.13, so revisit the figure then), and record the D: size and a disk-space warning threshold set the way the collector's is — a value that depends on your environment, recorded with the reason.
3. Fix the one way in (rule F11): SBD-SYSLOG01 writes to the LogArchive share over SMB 3 with encryption as SBD-Archive$, a group managed service account (gMSA) whose password no person knows. Nothing else connects to the share; people work on SBD-AUDIT01 itself, in sessions opened from SBD-ADMIN-SRV01 under rule A04.
4. Name every other identity and its one job: SBD-Catalog$ hashes arrivals, SBD-Retention$ is the only account the design lets delete, SBD Auditors read the record in remote sessions from SBD-ADMIN-SRV01, and SBD Audit Custodians (course 1.8) — the custodian's separate admin accounts — run the server but can't delete records either. In a Tier 0 session from AdminJumpBox01, create SBD Auditors under the Groups OU, add it to SBD-ADMIN-SRV01's seat GPO beside SBD Tier 1 Admins and SBD Audit Custodians (Allow log on locally, a change to course 1.8's STIG value that its exception record notes), and run Get-KdsRootKey on SBD-DC01. If it returns nothing, create the forest's one KDS root key with Add-KdsRootKey -EffectiveImmediately now: domain controllers wait up to ten hours before gMSAs can be created and used, so lesson 2 creates the accounts.
5. Build SBD-AUDIT01 (10.0.50.40, VLAN 50) as Windows Server with its data volume, DNS on both domain controllers, and time from the domain hierarchy (course 1.3): hypervisor time synchronization off, and w32tm /query /source naming a domain controller. Join it into a new Audit Servers OU (OU=Audit Servers,OU=SBD,DC=sbd,DC=test, created from AdminJumpBox01 with deletion protection) that already carries course 1.5's member-server STIG GPOs and course 1.8's two deny GPOs (the tier logon GPO and the network-logon deny GPO, never its Servers administrators GPO), so nothing granted to ordinary server administrators through the Servers OU reaches it. By GPO on that OU, make

SBD Audit Custodians the local Administrators in place of Domain Admins, as the STIG's member-server rule requires, put SBD Auditors in Remote Management Users, and scope Remote Desktop and Windows Remote Management to SBD-ADMIN-SRV01 (10.0.60.20). Its Windows updates come through SBD-WSUS01 once course 1.14 builds it, in a group of their own: downloaded only, and installed when the custodian starts them, never by ordinary Tier 1 operators.

Check your result

```
# On SBD-SYSLOG01 (Tier 1 session): one day of each stream, for sizing.
wevtutil epl ForwardedEvents $env:TEMP\sizing.evtx
"/q:*[System[TimeCreated[timediff(@SystemTime) <= 86400000]]]" /ow:true
(Get-Item $env:TEMP\sizing.evtx).Length / 1MB
(Get-ChildItem D:\Logs\Syslog -Recurse -File | Where-Object LastWriteTime -gt (Get-
Date).AddDays(-1) | Measure-Object Length -Sum).Sum / 1MB
# From AdminJumpBox01 (Tier 0): the KDS root key's effective time, and where SBD-AUDIT01
landed.
Invoke-Command SBD-DC01 { Get-KdsRootKey | Select-Object KeyId, EffectiveTime }
Get-ADGroup 'SBD Auditors' | Select-Object DistinguishedName
Get-ADComputer SBD-AUDIT01 | Select-Object DistinguishedName
# From SBD-ADMIN-SRV01 (custodian account): who administers SBD-AUDIT01.
Invoke-Command SBD-AUDIT01 { Get-LocalGroupMember Administrators; w32tm /query /source }
```

WHAT SUCCESS LOOKS LIKE The rules with their recorded retention periods, the sizing, and the identity table are written; the KDS root key exists with its effective time recorded; and SBD-AUDIT01 sits in the Audit Servers OU with SBD Audit Custodians in place of Domain Admins and management allowed from SBD-ADMIN-SRV01 alone, and its time comes from a domain controller.

Your record

Evidence to keep: Keep the repository rules and sizing sheet, the identity table (who writes, reads, deletes, and administers), the KDS root key's ID and effective time, and SBD-AUDIT01's OU placement and local Administrators list.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why does SBD-AUDIT01 get its own OU instead of joining the Servers OU?

> **REMEMBER** The repository keeps the record for the whole recorded retention period; the collector and analysis keep shorter working copies. One write path (F11), one writer, and one identity that can delete. SBD-AUDIT01 is built in its own OU and run only by the custodian, who also installs its updates.

Microsoft · Group managed service accounts

Microsoft · Create a Key Distribution Service (KDS) root key

Microsoft · Wevtutil

NIST · SP 800-53 Rev. 5 security controls (AU family)

LESSON 02 / ABOUT 40 MIN

Archive, hash, and protect the record

> **WHY THIS MATTERS** The repository is where the record outlives mistakes and attackers alike: daily archives from the collector over an encrypted share, hashed before they leave and again on arrival, readable by few, and deletable by process only.

In this lesson you will

- Archive the collector's two streams to the repository every day.
- Hash and catalog every archive on arrival.
- Enforce create-without-delete, read-few, and delete-by-process-only permissions.

Follow the build

1. From AdminJumpBox01 (Tier 0), create the three gMSAs now that the KDS root key is in effect, each usable by one computer only: New-ADServiceAccount SBD-Archive -DNSHostName SBD-Archive.sbd.test -PrincipalsAllowedToRetrieveManagedPassword 'SBD-SYSLOG01$', then SBD-Catalog and SBD-Retention the same way for 'SBD-AUDIT01$'. That allowed-computer list is each account's real access list: an administrator of that computer can act as the account, so it names one host.
2. On SBD-AUDIT01, create D:\LogArchive with ForwardedEvents, Syslog, Catalog, and Retention folders, remove its inherited permissions (icacls D:\LogArchive /inheritance:r), and grant, inheritable to everything below: SYSTEM full control; read and execute (RX) to SBD Audit Custodians, SBD Auditors, SBD-Catalog$, and SBD-Retention$; and OWNER RIGHTS read permissions only on files, '*S-1-3-4:(OI)(IO)(RC)', which takes away a file owner's implicit right to rewrite its permissions. On each stream folder, give SBD-Archive$ create files on that folder only — icacls D:\LogArchive\ForwardedEvents /grant 'SBD\SBD-Archive$:(WD,RA,S)' — and SBD-Retention$ delete on its files, 'SBD\SBD-Retention$:(OI)(IO)(DE)'. A file's creator gets the access it asks for while creating it, then only what the file inherits, so the writer can add archives but never read, replace, or delete them.
3. Share it: New-SmbShare -Name LogArchive -Path D:\LogArchive -EncryptData $true -ChangeAccess 'SBD\SBD-Archive$'. Share permissions are only the outer gate for network access — effective access is the stricter of share and NTFS — so the share names its one writer, and Get-SmbShareAccess must show nobody else, not even Everyone. Leave RejectUnencryptedAccess at its default (True) with SMB 1 off, so only encrypted SMB 3 sessions reach the share, and scope every enabled inbound rule for TCP 445 to 10.0.20.20: VLAN 50 will also hold SBD-SPLUNK01 (course 1.12) and the other security servers the System Hub places there, whose traffic to this server never crosses the core, so only this host firewall stops them.
4. On SBD-SYSLOG01 (Tier 1 work), register the archive task to run daily at 01:00 as SBD-Archive$: Register-ScheduledTask with New-ScheduledTaskPrincipal -UserId 'SBD\SBD-Archive$' -LogonType Password — no password is typed; Windows retrieves the managed one. Keep the script in a folder only administrators can change, because whoever can edit it acts as the writer. Through a GPO filtered to SBD-SYSLOG01 and linked above the STIG GPO, give SBD-Archive$ Log on as a batch job and add it to Event Log Readers with a Group Policy preference that adds a member rather than replacing the list (course 1.12 adds the Splunk forwarder's account to the same group the same way) — user rights don't merge across GPOs, so the list repeats any entries the STIG sets for that right — then grant it read on

D:\Logs\Syslog and a staging folder of its own, and send the grants to the owner of the course 1.5 exception record on AdminJumpBox01.

5. Each run, the script exports every completed UTC day not yet archived from ForwardedEvents into its staging folder — wevtutil epl with a TimeCreated filter for that day — and zips that day's syslog files, so a missed night is caught up while the collector's buffer still holds the events. It writes each file's SHA-256 (Get-FileHash) into a .sha256 file, copies the archive and then its .sha256 to the share, and keeps its own list of what it sent, because the writer can't list the share. When course 1.10's health check records a gap window, re-export the affected days once the source has replayed its backlog, under a new name such as ForwardedEvents-2026-10-12-r1.evtx: forwarded events keep their source's times, so the late ones land in the right day, and the first archive stays as it was.
6. On SBD-AUDIT01, register the catalog task the same way, hourly as SBD-Catalog$, with Log on as a batch job through the Audit Servers OU's policy and write, but not delete, on the Catalog folder and the files in it. For each archive whose .sha256 has arrived, it computes the SHA-256, appends a line to catalog.csv (columns Arrived, File, Size, SentSHA256, SHA256, Result), and writes the same line as an event from the source SBD-Repository (registered once with New-EventLog): 1000 when the hashes match, 1001 when they don't or a .sha256 is still missing after the wait you set and recorded — a finding, not a file. Then fill in the AU table: where each stage's data lives, its retention clock, and who can touch it — the kind of artifact course 2.2 shows joining the Body of Evidence for the audit controls.

Check your result

```
# On SBD-AUDIT01 (custodian session): the share, its encryption, its one writer, and a
stream folder's permissions.
Get-SmbShare -Name LogArchive | Select-Object Name, Path, EncryptData
Get-SmbShareAccess -Name LogArchive
Get-SmbServerConfiguration | Select-Object RejectUnencryptedAccess, EnableSMB1Protocol
icacls D:\LogArchive\ForwardedEvents
# On SBD-SYSLOG01 (Tier 1 session): the task's identity and its last result (0 = success).
(Get-ScheduledTask -TaskName 'SBD Archive to AUDIT01').Principal | Select-Object UserId,
LogonType
Get-ScheduledTaskInfo -TaskName 'SBD Archive to AUDIT01' | Select-Object LastRunTime,
LastTaskResult
# Back on SBD-AUDIT01: who wrote the newest archives, and their catalog lines.
Get-ChildItem D:\LogArchive\ForwardedEvents | Sort-Object Name | Select-Object -Last 2
Name, Length, @{n='Owner';e={(Get-Acl $_.FullName).Owner}}
Import-Csv D:\LogArchive\Catalog\catalog.csv | Select-Object -Last 4
```

> **WHAT SUCCESS LOOKS LIKE** Daily archives and their .sha256 files arrive owned by SBD-Archive$ on a share that requires encryption and refuses unencrypted access, every catalog line shows matching hashes, Get-SmbShareAccess lists the writer alone, and icacls shows delete rights for SYSTEM and SBD-Retention$ only.

Your record

Evidence to keep: Keep the share, encryption, and permission listings, the two task definitions with their gMSA principals, the first catalog lines, and the completed AU table.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	

RECORD	YOUR OBSERVATION
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

What does the hash catalog actually protect against?

> **REMEMBER** Archives arrive daily over one encrypted path, with two hashes that must agree. Readers are named; deletion is a process, not a right. The AU table is written for the authorization path.

Microsoft · Group managed service accounts

Microsoft · New-ScheduledTaskPrincipal

Microsoft · New-SmbShare

Microsoft · SMB security enhancements (SMB encryption)

Microsoft · Icacls

Microsoft · Special identity groups (Owner Rights)

Microsoft · Get-FileHash

Microsoft · Wevtutil

NIST · SP 800-53 Rev. 5 security controls (AU family)

LESSON 03 / ABOUT 30 MIN

Verify the catalog and watch the repository

> **WHY THIS MATTERS** A catalog nobody checks is a list of hopes. Re-hash every archive on a schedule, treat a mismatch as a finding, audit who reads or tries to change the repository, and keep the readers as few as the day they were approved.

In this lesson you will

- Verify every archive against the catalog on a schedule and report the result.
- Handle a mismatch as a finding, not a file to fix.
- Audit access to the repository and review who may read and write it.

Follow the build

1. Add a daily verification task on SBD-AUDIT01, running as SBD-Catalog$ like the catalog task: it re-hashes every archive and writes D:\LogArchive\Catalog\verify-<date>.csv with one row per file and its result — OK, MISMATCH, MISSING (cataloged but absent, or a day with no archive), or UNKNOWN (present but never cataloged). It then writes one event from SBD-Repository: 1100 when every row is OK, 1101 when any isn't.
2. Treat event 1101 as a finding: don't re-hash the file until it passes, edit its catalog line, or delete it. Record what failed and when it last verified, and read the repository's access events for that window. Re-export that day under a new name through the normal path while the collector's buffer still holds it — or, once course 1.19 has made them, recover an offline copy — keep the damaged file as evidence, and have the custodian report an incident.
3. Audit the repository itself: in a separate GPO linked to the Audit Servers OU, recorded as a documented addition with the exception record's owner, turn on Audit File System (success and failure) and Audit Detailed File Share (failure). On D:\LogArchive, add auditing entries for failed access of any kind by Everyone; successful delete, change permissions, and take ownership by Everyone; and successful reads by SBD Auditors and SBD Audit Custodians. File System auditing writes events only where those entries match, so the service accounts' routine work stays quiet.
4. Send that trail off the box: from AdminJumpBox01, add SBD-AUDIT01 to SBD Event Sources, link course 1.10's enrolling GPO to the Audit Servers OU, and restart SBD-AUDIT01 so both apply. On the collector, widen the WinRM 5985 scope to 10.0.50.40 (rule F17): its Security log, with events 4656, 4660, 4663, 4670, and 5145, now arrives through the subscription's Security query, so add only a query for its SBD-Repository events in the Application log, filtered to that computer. Someone who can alter SBD-AUDIT01's logs or its catalog still can't reach the copies on the collector, which other administrators run.
5. Review the people each quarter and after any change: compare SBD Auditors, SBD Audit Custodians, and SBD-AUDIT01's local Administrators with the approved list, and check that only SBD-SYSLOG01 may retrieve SBD-Archive$'s password — any computer added there could write as the archive job. Record the review with its date and reviewer.

Check your result

```
# On SBD-AUDIT01 (custodian session): the latest verification's exceptions, and the result events.
```

© 2026 Secure By Design Team LLC.

```
$report = Get-ChildItem D:\LogArchive\Catalog\verify-*.csv | Sort-Object Name | Select-
Object -Last 1
Import-Csv $report.FullName | Where-Object Result -ne 'OK'
Get-WinEvent -FilterHashtable @{LogName='Application'; ProviderName='SBD-Repository';
Id=1100,1101} -MaxEvents 3 | Select-Object TimeCreated, Id
# The audit policy and the auditing entries on the repository.
auditpol /get /subcategory:"File System"
auditpol /get /subcategory:"Detailed File Share"
(Get-Acl D:\LogArchive -Audit).Audit | Format-Table IdentityReference, FileSystemRights,
AuditFlags
# On SBD-SYSLOG01 (Tier 1 session): the repository's events arrive with their source name.
Get-WinEvent -FilterHashtable @{LogName='ForwardedEvents'; Id=4656,4663,1100,1101;
StartTime=(Get-Date).AddDays(-1)} -MaxEvents 5 | Select-Object TimeCreated, Id,
MachineName
# From AdminJumpBox01 (Tier 0): the readers, and the computers allowed to act as the
writer.
Get-ADGroupMember 'SBD Auditors' | Select-Object Name, SamAccountName
(Get-ADServiceAccount SBD-Archive -Properties
PrincipalsAllowedToRetrieveManagedPassword).PrincipalsAllowedToRetrieveManagedPassword
```

> **WHAT SUCCESS LOOKS LIKE** Today's report covers every archive and its event is 1100 (or an open 1101 is being handled as a finding), the audit policy and auditing entries are in place, the repository's events reach ForwardedEvents on the collector with MachineName SBD-AUDIT01, and the readers review is recorded.

Your record

Evidence to keep: Keep a verification report with its event, the audit policy and auditing-entry listings, one forwarded repository event, and the dated readers review.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

The daily verification reports a MISMATCH. What is the right first move?

> **REMEMBER** Every archive is re-hashed daily, and the result is an event. A mismatch is preserved and investigated, never re-hashed away. Access to the repository is audited, forwarded, and reviewed.

Microsoft · Group managed service accounts

Microsoft · Get-FileHash

Microsoft · Advanced audit policy configuration (File System, Detailed File Share)

Microsoft · Use Windows Event Forwarding to help with intrusion detection

NIST · SP 800-53 Rev. 5 security controls (AU family)

LESSON 04 / ABOUT 30 MIN

Run retention and prove retrieval

> **WHY THIS MATTERS** Retention cuts both ways: records must last the full period, and they must go when it ends — by a process with a log, never by a person's right-click. Run retention the way the design says, then prove the other half: an archive from the past can be found, verified, and read.

In this lesson you will

- Run retention through an approved, logged process under its own identity.
- Show that nothing younger than the retention period can be removed.
- Retrieve a past archive, verify its hash, and read its events.

Follow the build

1. Build the retention task on SBD-AUDIT01 to run as SBD-Retention$ (batch logon, like SBD-Catalog$) with no trigger, so it runs only when the custodian starts it. It reads the newest approved list in D:\LogArchive\Retention, re-verifies each listed archive against the catalog, refuses any file younger than the recorded retention period by its catalog arrival date — whatever the list says — and deletes the rest with their .sha256 files. It appends one line per file to retention-log.csv (date, file, catalog hash, list hash, result), which the daily verification now reads, so a retired archive reports RETIRED rather than MISSING. SBD-Retention$ gets write on the Retention folder and the custodians create files there only, so a list can be added but not edited.
2. Run it as a two-person process, the independent approval rule A04 asks for (AU-9(5), dual authorization): the custodian prepares the list of archives past retention, an approver who is not a custodian approves it and records the list's SHA-256, and the custodian starts the task with Start-ScheduledTask. The log carries the same list hash, tying the run to the approval, and every deletion also writes events 4663 and 4660 naming SBD-Retention$, which reach the collector.
3. Prove the guard now: nothing in a new lab is past its retention period, so put one archive from lesson 2 on an approved list marked as a test. The run must refuse it, log the refusal, and leave the file and its catalog line untouched.
4. Prove retrieval: pick the oldest archive in the catalog, confirm its SHA-256 still matches its catalog line, and read it — Get-WinEvent -Path for a ForwardedEvents archive, or expand a syslog ZIP into D:\Retrieval, a working folder outside the archive, and search it. Security events render because SBD-AUDIT01 has the same Windows publishers; the event data itself is in the file either way.
5. Record the retrieval — the archive, both hashes, the events you read, who, and when — and repeat it each quarter with an older archive, so over time you prove retrieval across the whole retention period. Course 1.19 retrieves the offline copies the same way.

Check your result

```
# On SBD-AUDIT01 (custodian session): no trigger, so only a person starts a run.
(Get-ScheduledTask -TaskName 'SBD Repository Retention').Triggers.Count   # expected 0
(Get-ScheduledTask -TaskName 'SBD Repository Retention').Principal.UserId   # expected:
the SBD-Retention$ account
Get-FileHash D:\LogArchive\Retention\approved-test-list.txt   # the hash the approver
recorded
Start-ScheduledTask -TaskName 'SBD Repository Retention'
Import-Csv D:\LogArchive\Retention\retention-log.csv | Select-Object -Last 5
```

```
# Retrieval: the oldest archive's hash against its catalog line, then the events inside.
$file = Get-ChildItem D:\LogArchive\ForwardedEvents\*.evtx | Sort-Object Name | Select-
Object -First 1
(Import-Csv D:\LogArchive\Catalog\catalog.csv | Where-Object File -eq $file.Name).SHA256 -
eq (Get-FileHash $file.FullName).Hash
Get-WinEvent -Path $file.FullName -MaxEvents 5 | Select-Object TimeCreated, Id,
MachineName
```

> **WHAT SUCCESS LOOKS LIKE** The retention task has no trigger and runs as SBD-Retention$, the approved test run refused the young archive and logged the refusal with the list's hash, and the retrieved archive's hash matches its catalog line and its events read correctly.

Your record

Evidence to keep: Keep the retention task definition, the approved test list with its hash and approval, the retention log, and the retrieval record with both hashes and a sample of the events read.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why does the retention task have no schedule?

> **REMEMBER** Retention runs only on an approved list, under its own identity, with a log. The task refuses anything younger than the retention period. A past archive was found, verified, and read — and will be again each quarter.

Microsoft · New-ScheduledTaskPrincipal

Microsoft · Get-FileHash

Microsoft · Get-WinEvent

NIST · SP 800-53 Rev. 5 security controls (AU family)

LESSON 05 / ABOUT 30 MIN

Prove the record survives and hand off

> **WHY THIS MATTERS** Test the protection the way an attacker or a careless administrator would: try to delete an archive and tamper with a copy, and check that each attempt is refused or caught, and recorded. Then take SBD-AUDIT01's first baseline and hand the record to analysis.

In this lesson you will

- Show that an administrator's delete attempt is refused and logged on and off the repository.
- Show that a tampered copy fails verification against the catalog.
- Baseline SBD-AUDIT01 and write the handoff to course 1.12.

Follow the build

1. In a custodian session on SBD-AUDIT01 — an administrator of this server — try to delete the newest archive with Remove-Item. It must fail with access denied: administrators hold read only, and only SBD-Retention$ can delete. An administrator can still force their way in — by taking ownership, using backup and restore privileges, or running a process as SYSTEM — which is why only the custodians administer this server and those steps are audited too. Use the newest archive, so if the delete wrongly succeeds that day can be re-exported from the collector's buffer — and you have a finding to fix first.
2. Find the record of the attempt: event 4656, an audit failure naming your account, the archive, and DELETE, in SBD-AUDIT01's Security log — then the same event in ForwardedEvents on SBD-SYSLOG01 with MachineName SBD-AUDIT01. The attempt and its refusal now exist on two systems with different administrators.
3. Tamper with a copy, never the record: copy the newest archive to D:\Retrieval\tamper-test, append one byte to the copy, and compare its SHA-256 with the catalog line. The copy must fail while the record still matches — the same check that would catch an altered offline copy in course 1.19.
4. Take SBD-AUDIT01's first SCAP baseline with the course 1.4 SCC and benchmark versions, run locally in the custodian's session: this server refuses SMB from VLAN 60, so a remote scan can't reach it, and the installer arrives from SBD-ADMIN-SRV01's course 1.8 copy over PowerShell remoting (Copy-Item -ToSession) with its course 1.4 hash checked. Store the results under C:\Evidence\audit-repository with hashes, and classify open items as course 1.4 taught — fix by GPO, deviation candidate, or POA&M candidate — noting this course's documented additions: batch-logon rights and object-access auditing.
5. Write the handoff to course 1.12: the repository keeps the protected copy and stays out of the analysis flow, which takes its working copy from the collector over F10; a day's events reach the repository in the next morning's archive, which is where an end-to-end trace finds them; the repository's events — 1001 and 1101 findings, refused access, and retention deletions — are ready to become alerts; and course 1.19 sets up the backup of the archive and its catalog, which the audit custodian carries to the backup cabinet.

Check your result

```
# Custodian session on SBD-AUDIT01: try to delete the newest archive (expected: access
denied).
$newest = Get-ChildItem D:\LogArchive\ForwardedEvents\*.evtx | Sort-Object Name | Select-
Object -Last 1
Remove-Item $newest.FullName
```

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4656; StartTime=(Get-
Date).AddMinutes(-15)} | Where-Object KeywordsDisplayNames -contains 'Audit Failure' |
Select-Object -First 3 TimeCreated, Id
# On SBD-SYSLOG01 (Tier 1 session): the same refusal, forwarded.
Get-WinEvent -FilterHashtable @{LogName='ForwardedEvents'; Id=4656; StartTime=(Get-
Date).AddHours(-1)} | Select-Object TimeCreated, MachineName
# Back on SBD-AUDIT01: a tampered copy fails the catalog check while the record passes.
New-Item -ItemType Directory D:\Retrieval\tamper-test -Force | Out-Null
Copy-Item $newest.FullName D:\Retrieval\tamper-test\
Add-Content "D:\Retrieval\tamper-test\$($newest.Name)" -Value 'x' -NoNewline
$expected = (Import-Csv D:\LogArchive\Catalog\catalog.csv | Where-Object File -eq
$newest.Name).SHA256
(Get-FileHash "D:\Retrieval\tamper-test\$($newest.Name)").Hash -eq $expected   # expected
False
(Get-FileHash $newest.FullName).Hash -eq $expected   # expected True
Get-FileHash -Algorithm SHA256 C:\Evidence\audit-repository\*
```

WHAT SUCCESS LOOKS LIKE The delete failed and appears as event 4656 on SBD-AUDIT01 and in ForwardedEvents on the collector, the tampered copy fails verification while the record passes, SBD-AUDIT01 has its first baseline, and the handoff names what course 1.12 inherits.

Your record

Evidence to keep: Keep the refused delete with both events, the tamper test's hash comparison, the scan set with hashes, and the handoff note to course 1.12.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

The custodian administers SBD-AUDIT01. Why can't their admin account delete an archive?

REMEMBER An administrator's delete was refused and recorded on two systems. A tampered copy failed verification; the record still passes. SBD-AUDIT01 is baselined, and course 1.12 inherits a protected record.

Microsoft · Get-FileHash

Microsoft · Advanced audit policy configuration (File System, Detailed File Share)

Microsoft · Get-WinEvent

DISA · SRG/STIG tools (SCC and STIG Viewer)

NIST · SP 800-53 Rev. 5 security controls (AU family)

Course review and sign-off

Before moving on, confirm each lesson left evidence behind — and that what remains open is written down, not remembered.

Lesson completion

LESSON	EVIDENCE FILED	DATE
01 Design the repository and build SBD-AUDIT01	☐ yes	
02 Archive, hash, and protect the record	☐ yes	
03 Verify the catalog and watch the repository	☐ yes	
04 Run retention and prove retrieval	☐ yes	
05 Prove the record survives and hand off	☐ yes	

Open items carried forward

Next in the learning path

That completes course 1.11. Course 1.12 builds analysis in SBD-SPLUNK01 on a working copy from the collector and turns the repository's findings into alerts.

> **KEEP IT HONEST** Lesson completion records learning activity; it is not a system authorization decision. Your observations, not the sample values, establish what happened in your environment.

© 2026 Secure By Design Team LLC.